# Execution Semantics Are Statistics: Rules for Valid Security Evaluation of LLM Agents on Unreliable Infrastructure

[AUTHORS AND AFFILIATIONS TO BE INSERTED]

Working manuscript v2, 18 October 2026 — simulation results only; real-agent results pending

**Abstract**

**Purpose.** Security evaluations of LLM agents confirm vulnerabilities statistically, by repeating stochastic agent episodes, and run them on distributed harnesses whose defaults—at-least-once delivery, retry with fresh randomness, results consumed as they arrive, worker-local configuration—were designed for throughput. We ask whether these execution semantics preserve the error guarantees of the reported results. **Methods.** We state six execution rules and prove that, under them, both sequential (e-process) and fixed-sample (exact binomial) confirmation keep their nominal false-confirmation level under message loss, duplication, reordering, crashes, partitions and elastic membership. We show by construction that each corresponding default can break validity, and evaluate the rules in a deterministic fault-injection simulator and slot-level Monte Carlo with known ground truth. **Results.** At nominal level 0.05, executor defaults raised the false-confirmation probability of sequential confirmation to 0.08–0.13 and of fixed-sample confirmation to 0.11–0.999; with the rules it stayed at or below 0.035 in every configuration. A reference implementation scaled 25× on 32 workers at about 3.5 messages per episode. Auditing the counterfactual-margin estimator of MARGIN-Bench, we found it exceeds its nominal error (0.257) and give an exact repair. A shared coordinator gave no speed-up over independent runs with a split budget, and anytime-valid confirmation cost 2–5 points of power against the repaired fixed design. **Conclusion.** Execution semantics belong in the statistical protocol of agent security evaluation. Real-agent measurements are specified but not yet executed.

**Keywords:** LLM agents; security evaluation; statistical validity; distributed systems; fault tolerance; sequential testing; e-values; reproducibility

## 1 Introduction

Tool-using language-model agents fail securely or insecurely through *actions*—an unauthorized ledger update, a forwarded secret, a privilege change—and benchmarks increasingly measure these actions in executable sandboxes [1–4]. Because agents are stochastic, a single violating episode is weak evidence; best-of-$N$ attacks show that success grows with repetition [5]. Rigorous evaluations therefore *confirm* vulnerabilities statistically: a candidate intervention is declared to violate policy with probability at least $\alpha$ only when repeated episodes support it at a stated error level [6, 7]. MARGIN-Bench [8] builds a full estimand on this idea, the counterfactual safety margin

$$R_{s,\alpha}(M, E) = \inf\{\, c_s(\delta) \;:\; p_M(E, \delta) \geq \alpha \,\}, \tag{1}$$

the cheapest intervention on attack surface $s$ whose violation probability reaches $\alpha$.

Such evaluations need $10^7$–$10^8$ agent episodes and run on distributed harnesses. The defaults of those harnesses are reasonable for throughput and harmful for statistics. A queue that redelivers a result makes one episode count twice. A retry policy that re-runs crashed episodes with

fresh randomness keeps the outcomes of episodes that happened not to crash—and long, benign trajectories crash more often. Consuming results as they arrive orders evidence by latency, which depends on the outcome. Letting each worker apply its own error budget multiplies the family-wise error by the number of workers. None of these is a bug in the statistical test; each changes the data the test sees.

This paper argues that *execution semantics are part of the statistical protocol* and makes the argument precise.

**Contributions.**

C1 **Six execution rules** (Table 1) and proofs that, under them, sequential e-process confirmation (Theorem 1) and fixed-sample exact confirmation (Proposition 2) keep their nominal error under message loss, duplication, reordering, crashes, partitions and elastic membership, for any scheduler.

C2 **Necessity.** For each rule, the corresponding executor default admits executions whose false-confirmation probability exceeds the nominal level, in several cases tending to one (Theorem 3); we reproduce this for both test families in simulation.

C3 **Audit and repair** of the MARGIN-Bench margin estimator, which as specified exceeds its nominal error five-fold; an exact repair restores validity at no efficiency cost.

C4 **Cost of the rules.** A reference implementation (a single-ledger protocol) and a fault-injection evaluation showing near-linear scaling, small coordination cost and fast recovery.

C5 **Estimator guidance** from preregistered comparisons: a repaired fixed design is the most powerful at a fixed budget; anytime-valid confirmation buys flexible stopping at a 2–5 point power cost; a shared coordinator is unnecessary for speed.

**Non-claims.** We do not propose a new statistical test; the inferential results we rely on are standard. We claim no certified lower bound on the margin, no Byzantine tolerance, validity only when episodes share no hidden state, and no finding about real agents: all results are synthetic, and the real-agent protocol (Section 8.7) is specified but not executed.

## 2 Setting

**Confirmation.** A generator proposes candidates $\delta_1, \delta_2, \ldots$ with costs $c(\delta_j) \in \{1, \ldots, K\}$. Executing candidate $j$ in *slot* $k$ with randomness key $u_{j,k}$ yields the committed-violation label $Y_{j,k} = Y(\delta_j, u_{j,k}) \in \{0, 1\}$ of a permission-grounded oracle; $p(\delta_j) = \Pr_u[Y = 1]$. A *confirmation procedure* declares candidate $j$ confirmed when its evidence rejects $H_j : p(\delta_j) \le \alpha$ at level $a_j$; the reported margin is $\hat{b} = \min\{c(\delta_j) : j \text{ confirmed}\}$ and the target is

$$\Pr(R_{s,\alpha} > \hat{b}) \le \varepsilon, \tag{2}$$

implied by the no-false-witness property $\Pr(\exists j \text{ confirmed} : p(\delta_j) \le \alpha) \le \varepsilon$. We consider two families: *fixed-sample* confirmation (an exact binomial test on a predetermined number $n$ of slots) and *sequential* confirmation (a betting e-process that may stop at any slot).

**The harness.** Workers request slots, executors run one sandboxed episode per slot from an immutable snapshot, and results travel over a network that may delay, reorder, duplicate or drop messages and partition. Workers and executors fail by crashing; membership changes over time. Workers are honest (they belong to the evaluator); Byzantine behaviour is out of scope. The generator is treated as untrusted with respect to the statistics: it sees committed outcomes but never a slot's key before the slot is registered.

**Assumption 1** (Independent episodes)**.** *Keys are i.i.d. and independent of the generator's randomness, and episodes share no hidden state, so $Y_{j,k}$ given the registration of $j$ is Bernoulli$(p(\delta_j))$ independently across slots.*

**Assumption 2** (Crash independence)**.** *Whether a slot crashes may depend on its own trajectory and on infrastructure noise independent of other slots' keys. (Not needed when crashed slots are replayed with the same key.)*

**Assumption 3** (Oracle correctness)**.** *$Y$ is the true committed-violation label; with false-positive rate $\omega$ all guarantees concern $p + \omega(1 - p)$.*

## 3 Related work

**Valid inference under adaptivity.** Test supermartingales, Ville's inequality and e-values give inference valid under optional stopping [9–11]; betting confidence sequences [12, 13], always-valid A/B testing [14], e-value and online multiple testing [15–17], weighted error spending [18] and group-sequential designs with delayed responses [19] are mature. All assume the observation sequence is well defined. We study how a fault-prone executor *produces* that sequence.

**Statistical assurance in testing and search.** Residual-risk estimation in fuzzing [20, 21], statistical model checking [22, 23], scenario optimisation [24], threshold and infinitely-armed bandits [25–28] and their parallel variants [29, 30] bound errors of search or identification; parallel fuzzing schedulers remove redundant work for deterministic oracles [31]. For stochastic confirmation, redundancy and ordering become correctness issues.

**Distributed systems.** We reuse leases [32], escrow [33], monotone state and coordination-freedom [34, 35], causal ordering [36, 37], list-scheduling bounds [38] and task frameworks [39]. Treating a non-replenishable statistical budget as a cluster resource has a precedent in differential-privacy budget scheduling [40].

**LLM agent security.** Benchmarks cover injection, tool use, long-horizon and multi-agent attacks [2–4, 41–43]; counterfactual and causal analyses separate permission from apparent risk [44, 45]; defences enforce capabilities, provenance and delegation [46–49]; time-of-check-to-time-of-use flaws affect agents themselves [50]. We are not aware of work treating the evaluation harness's execution semantics as part of the statistical protocol.

**What is new.** Each mechanism in our rules (idempotent delivery, in-order commit, deterministic replay, leases) is established, and so is each inferential result. The contribution is the mapping between them: which executor semantics preserve or break which test assumptions, proofs for both test families, constructions showing necessity, a quantified audit of a concrete estimator, and measurements of what the rules cost.

## 4 Execution rules

**Theorem 1** (Sequential confirmation under the rules)**.** *Let candidate $j$ use the e-process $E_{j,n} = \prod_{i \le n}(1 + \lambda_{j,i}(\tilde{Y}_{j,i} - \alpha))$ with $\lambda_{j,i} \in [0, 1/\alpha)$ a function of registration data and $\tilde{Y}_{j,1..i-1}$, and be confirmed when $\sup_n E_{j,n} \ge 1/a_j$, where $a_j$ is fixed at registration and $\sum_j a_j \le \varepsilon$ almost surely. Under Assumptions 1–2 and rules R1–R5, for every generator, scheduler, stopping rule and execution order,* $\Pr(\exists j : H_j \text{ true}, j \text{ confirmed}) \le \varepsilon$.

*Proof.* Extend each candidate's slot sequence to infinity and let $E^*_{j,n}$ be the e-process on the full recorded sequence, $\mathcal{G}_{j,n} = \sigma(\text{registration of } j, \tilde{Y}_{j,1..n}, \text{crash noise of slots } 1..n)$. By R1 and

**Table 1:** Execution rules, the default each one forbids, and the test families that need it.

| Rule | Forbidden default | Needed by |
|---|---|---|
| **R1 Fresh index-keyed slots.** Key $u_{j,k} = \mathrm{PRF}_\kappa(\mathrm{run}, j, k)$, revealed only when the slot is leased. | seed sets shared across candidates and reused for selection; seeds visible to the generator | both |
| **R2 Predictable decisions.** Bets use only registration data and the committed prefix; fixed-sample tests run only on the complete slot set. | betting on or peeking at results not yet committed | both |
| **R3 Exactly-once, index-determined evidence.** Each slot has one terminal value (first write wins, fenced by a lease token); evidence is the in-order committed prefix (sequential) or the predetermined index set (fixed-sample). | at-least-once consumption; consuming results in arrival order; "first $n$ results to arrive" | both |
| **R4 Direction-aware imputation, no fresh retry.** A crashed slot counts as non-violating for confirmation (violating for audits); re-running is allowed only with the same key. | retry on failure with fresh randomness | both |
| **R5 One global error budget.** Every level $a_j$ is spent from a single budget, e.g. leases from one ledger or a fixed split $\varepsilon/W$. | each worker spending $\varepsilon$ on its own | both |
| **R6 Verifiable confirmation.** Decisions are recomputed from stored slot records, not taken from workers. | trusting worker-side decisions | both (integrity) |

Assumptions 1–2, $u_{j,n+1}$ is independent of $\mathcal{G}_{j,n}$; by R4, $\tilde{Y} \le Y$ pointwise; so $\mathbb{E}[\tilde{Y}_{j,n+1} \mid \mathcal{G}_{j,n}] \le p(\delta_j) \le \alpha$ under $H_j$, and with R2 $E_j^*$ is a non-negative supermartingale from 1. Ville's inequality conditional on registration gives $\Pr(\sup_n E_{j,n}^* \ge 1/a_j) \le a_j$. By R3 the committed stream is a prefix of the full sequence, so any confirmation implies $\sup_n E_{j,n}^* \ge 1/a_j$ whatever the schedule, stopping or abandonment. Summing over registrations and using R5 gives the bound $\varepsilon$. □

**Proposition 2** (Fixed-sample confirmation under the rules)**.** *Let candidate $j$ be confirmed when $S_j = \sum_{k=1}^n \tilde{Y}_{j,k} \ge s_j^*$, where the index set $\{1, \dots, n\}$ and the level $a_j$ are fixed at registration, $\Pr(\mathrm{Bin}(n, \alpha) \ge s_j^*) \le a_j$ (equivalently, the exact Clopper–Pearson lower bound exceeds $\alpha$), the test is evaluated only when all $n$ slots are terminal, and $\sum_j a_j \le \varepsilon$. Under Assumptions 1–2 and rules R1–R5 the same guarantee holds.*

*Proof.* Given registration, $(Y_{j,k})_{k \le n}$ are i.i.d. Bernoulli$(p(\delta_j))$ by R1 and Assumption 1; R3 makes $S_j$ the sum over exactly these indices, each counted once; R4 gives $\tilde{Y} \le Y$; so $S_j$ is stochastically dominated by $\mathrm{Bin}(n, p(\delta_j))$, which under $H_j$ is dominated by $\mathrm{Bin}(n, \alpha)$. Hence $\Pr(S_j \ge s_j^*) \le a_j$, and the union bound with R5 gives $\varepsilon$. R2 forbids evaluating the test on partial sets, which would turn it into an uncorrected sequential test. □

**Theorem 3** (Necessity of the rules)**.** *For each rule there is a fault pattern, with all other rules kept, under which the false-confirmation probability exceeds the nominal level:*

1. R3, duplicates. *Sequential: every result applied twice yields $(E^*)^2$, and at $p = \alpha$ the error lies in $[\sqrt{\varepsilon}/(1 + \lambda(1-\alpha)), \sqrt{\varepsilon}]$. Fixed-sample: duplicates displace fresh slots, shrinking the effective sample and inflating the variance of $S_j/n$.*

2. R4, fresh retry. *If benign episodes crash with probability $c_0 > c_1$ (the violating rate), the recorded stream is Bernoulli$(p(1-c_1)/(p(1-c_1)+(1-p)(1-c_0))) > \alpha$ at $p = \alpha$; a consistent sequential test then errs with probability tending to one, and a fixed-sample test errs with probability increasing in $n$.*

3. R3, arrival order. *If violating episodes return first, the first $n$ arrivals out of $m > n$ launched, or the arrival-ordered prefix, over-represent violations; as $m \to \infty$ the error tends to one for both families.*

4. R5. *$W$ workers each spending $\varepsilon$ give family-wise error $1 - (1 - \varepsilon_0)^W \to 1$.*

5. R1/R2. *Selecting candidates on a shared fixed seed set and confirming on the same seeds, or betting with knowledge of the current outcome, yields error tending to one.*

*Proof.* Each item is a direct construction; items 1 (sequential) and 2 follow from the stated distributions and optional stopping, item 3 from the order statistics of latencies, item 4 from independence across workers. Section 8 measures each. □

**Remarks.** (i) Theorem 1 decouples validity from scheduling: any allocation heuristic, learned or distributed, is safe under R1–R5. (ii) Under drift both results hold for the null "$p \leq \alpha$ at every slot". (iii) Replaying a crashed slot with the same key reproduces its outcome, removing the power loss of R4 and the need for Assumption 2; it requires seeded, deterministic execution, which must be audited.

## 5 Confirmation procedures

**MARGIN-Bench Algorithm 1 and its repair.** The MARGIN-Bench estimator processes budget levels in ascending order; at each level it screens $q$ candidates with $r_d$ discovery trials, sends the top $T$ to confirmation on $r_h$ hold-out trials, and returns the first confirmed level, under "a predefined confidence and multiplicity rule". The natural reading—a Wilson bound at level $\varepsilon/T$—corrects within a level but not across the $K$ levels tested in sequence, and the Wilson bound is anti-conservative at $r_h = 30$. The repair tests each of the $KT$ confirmations with an exact Clopper–Pearson bound at $\varepsilon/(KT)$, a fixed-sample procedure covered by Proposition 2; repeated passes use $\varepsilon/(n(n+1)KT)$ in pass $n$.

**LEXIS and LEXIS-S.** LEXIS opens candidates one at a time per level and confirms each with a plug-in Kelly e-process capped at $m$ trials, with weights $a_j$ from a polynomial schedule (lease $n$ receives $\varepsilon/(n(n+1))$, split over $R$ registrations). LEXIS-S adds Algorithm 1's screening: scouting trials on keys independent of confirmation keys, promotion of the top $T$ with weights and bet priors fixed from scouting data, and capped e-process confirmation; Theorem 1 applies because scouting data are registration information. Both are anytime-valid: they can be stopped at any time or extended with more passes without re-planning.

**Certificates.** Searched budgets that yield no confirmation can be summarised by anytime-valid bounds on the generator's violating mass per level (a zero-failure bound $1 - \beta^{1/N}$ and a betting upper confidence sequence; R4 imputes crashed audits as violating). These bound the generator's distribution, never the margin itself.

## 6 Reference implementation

The rules can be met in several ways; any harness that satisfies Table 1 inherits the guarantees. Our reference implementation (Fig. 1) uses one logical ledger as the serialisation point. Its state (candidates, slots, budget leases, frontier, audits) changes only through journaled, atomic transitions—Grant, Register, Lease, Record, Expire, Advance, Prune, Close, Audit. Requests carry (sender, request id) with cached responses, so retransmissions are idempotent; slot results are fenced by lease tokens (R3); expired slots become non-violating records (R4) or, in replay mode, are re-leased with the same key; budget leases are bound to a worker incarnation and

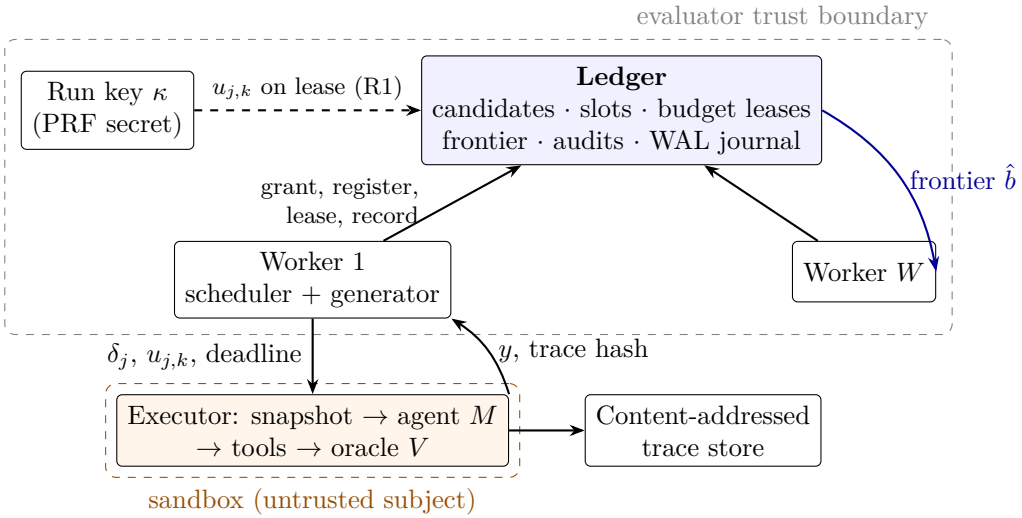



**Figure 1:** Reference implementation. A single logical ledger enforces R1, R3, R5 and R6; workers and the frontier broadcast are soft state that affects only efficiency.

fenced on close (R5); and confirmation is recomputed from stored slots (R6). Invariants (budget conservation, write-once slots, monotone commit pointer and frontier, confirmation soundness) are checked after every transition in property-based tests, and journal replay reproduces the ledger state exactly after a coordinator crash.

**Algorithm 1** Worker loop. Any scheduling policy is valid under Theorem 1.

**loop**
  **if** no open budget lease **then**
    request GRANT
  **end if**
  $a \leftarrow$ SCHEDULE(latest frontier, local state)
  **if** $a = \text{NEW}(k)$ **then**
    $\delta \leftarrow G_k$.PROPOSE(); REGISTER(lease, $\delta$) ▷ $a_j$ fixed here
  **else if** $a = \text{CONTINUE}(j, m)$ **then**
    **for** each $(k, u, \text{tok}) \in \text{LEASE}(j, m)$ **do**
      $y \leftarrow \text{EXECUTE}(\delta_j, u, \text{deadline})$; **if** no crash **then** RECORD$(j, k, y, \text{tok})$ ▷ retried until acknowledged
    **end for**
  **end if**
**end loop**

A shared ledger is not the only way to satisfy R5: $W$ independent single-worker deployments with budget $\varepsilon/W$ each also comply. Section 8 compares the two. Production deployments need no message broker, cache or custom consensus—a broker would add an at-least-once hop that must then be deduplicated—and task frameworks may place executors only with automatic retries disabled, since those retries implement the fresh-retry default.

## 7 Evaluation methodology

**Why synthetic.** Validity claims need ground truth—which candidates are null and the true margin $k^*$—which real agents never provide. We use (a) a deterministic discrete-event simulator that runs the implemented ledger, message layer and workers with injected faults (loss, duplication, reordering, partitions, worker and executor crashes, ledger crash and journal recovery,

outcome-correlated lognormal latency with means 20 s benign and 30 s violating) and Bernoulli oracles of known $p$; and (b) slot-level Monte Carlo of the confirmation procedures.

**Instances.** Four cost levels; S1-moderate has violating fractions $(0, 0, 0.05, 0.10)$ with $p = 0.6$, S2-sparse $(0, 0, 0.01, 0.05)$, and nulls with $p \sim U(0, 0.2)$; the boundary null sets every candidate to $p = \alpha = 0.2$, so every confirmation is false. $\alpha = 0.2$, $\varepsilon = 0.05$, four executors per worker.

**Statistics.** Seeds are the unit of analysis. Validity is judged by Wilson intervals against $\varepsilon$; time-to-margin is censored at a common horizon and compared by restricted mean survival time with bootstrap intervals and Holm-corrected Mann–Whitney tests [51]; non-inferiority uses Newcombe intervals for differences of proportions. Hypotheses and decision criteria were committed before the full runs (Table 4); the fixed-sample rule study (Table 2) was added during this reframing and is not preregistered.

# 8 Results

All results are synthetic.

## 8.1 The rules matter for both test families

Through the full system (400 seeds per cell, boundary null, sequential confirmation), removing deduplication raised the false-confirmation probability to 0.133 [0.103, 0.169], allowing fresh-key retry to 0.102 [0.076, 0.136], and consuming results in arrival order to 0.083 [0.059, 0.114]; with all rules it stayed between 0.003 and 0.018 in every fault configuration, including all faults combined (0.003 [0.000, 0.014]). A worker-local budget at $W = 8$ gave 0.065 [0.045, 0.094], an inflation not established at this configuration. For fixed-sample confirmation (Table 2) the defaults were at least as harmful: using the first $n$ results to arrive broke validity almost surely (0.999), fresh retry gave 0.189, duplicates 0.111, and a worker-local budget at $W = 8$ 0.266; with the rules the rate stayed at or below 0.035.

**Table 2:** Fixed-sample (exact Clopper–Pearson) confirmation of a 12-test family at the boundary null, $n = 30$, level $\varepsilon/12$ per test (50,000 runs; 95% Wilson intervals). Not preregistered.

| Configuration | Pr(false confirmation) |
|---|---|
| Rules applied, no faults | 0.035 [0.034, 0.037] |
| Rules applied, crashes 0.3 / 0.1 imputed as non-violating | 0.014 [0.013, 0.015] |
| Rules applied, $W = 8$ with one global budget | 0.019 [0.016, 0.023] |
| Default: duplicates counted (rate 0.3) | 0.111 [0.108, 0.114] |
| Default: crash → fresh-key retry | 0.189 [0.186, 0.193] |
| Default: first $n$ of $2n$ arrivals, violations faster | **0.999** [0.998, 0.999] |
| Default: worker-local budget, $W = 8$ | 0.266 [0.255, 0.277] |

## 8.2 Audit of the MARGIN-Bench estimator

As specified, Algorithm 1 confirmed a boundary-null candidate in 0.257 [0.238, 0.276] of 2,000 runs, five times its nominal level (Fig. 2); the exact repair gave 0.035 [0.027, 0.043] with indistinguishable power (P(reach $k^*$) 0.800 versus 0.803 on S1).

## 8.3 What the rules cost

The reference implementation reached the true margin 25.4× faster with 32 workers than with one on S1 (parallel efficiency 0.79; at least 21.4× on S2, where single-worker runs were often

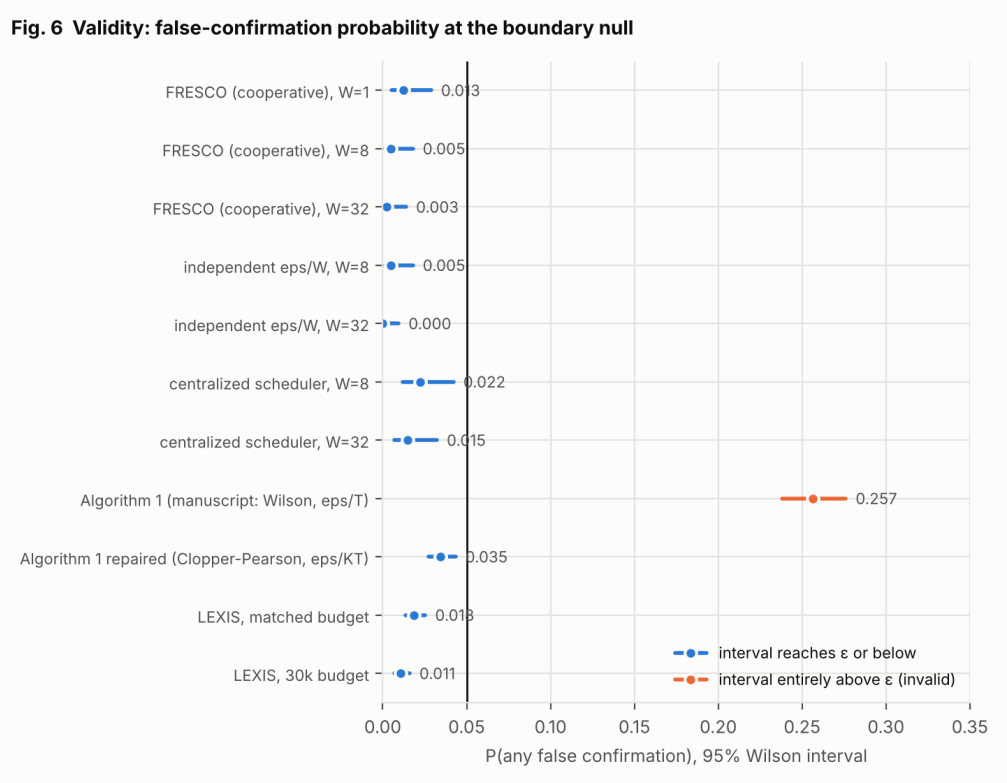



**Figure 2:** False-confirmation probability at the boundary null for all procedures and deployments. Only Algorithm 1 as specified is invalid.

censored) while the number of episodes needed stayed nearly flat (5.2k to 6.6k, overlapping intervals; Fig. 3). Coordination cost about 3.3–3.7 messages per episode and roughly 10 ledger operations per second at $W = 32$ in virtual time. After a 60 s ledger crash, a crash of half the workers or a 300 s partition, throughput recovered within one 20 s measurement bucket, and lost work matched the arithmetic loss. The main efficiency cost of the rules is conservative crash imputation: under 20% crashes, time to the margin rose 1.7×, mostly recovered by same-key replay.

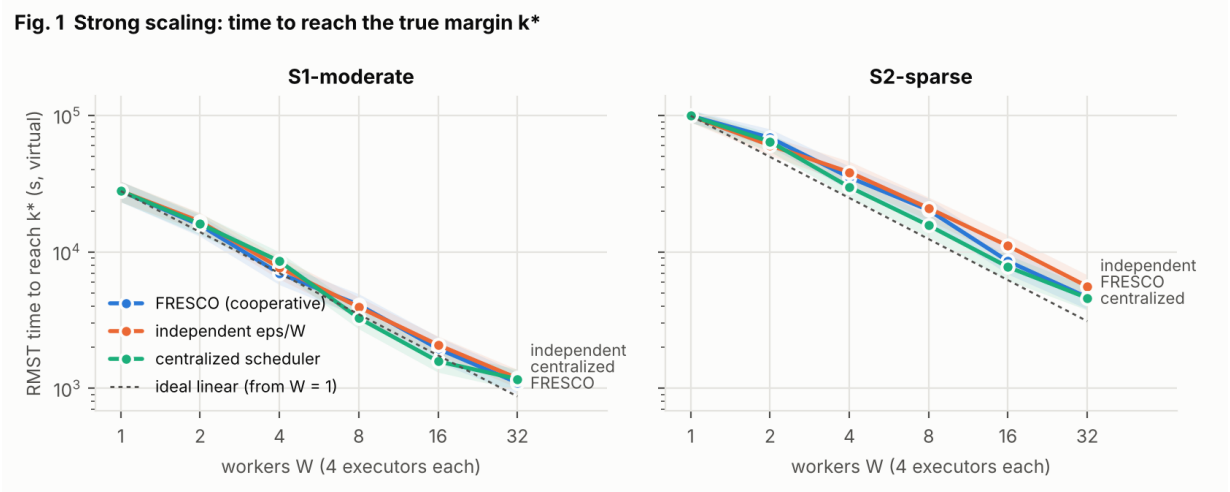



**Figure 3:** Time to reach the true margin versus workers for the shared-ledger deployment, $W$ independent deployments with budget $\varepsilon/W$, and one centralized scheduler (shaded: 95% bootstrap CI). All three comply with the rules.

### 8.4 Choosing a confirmation procedure

At a matched budget of 1,160 episodes (Table 3), repaired Algorithm 1 reached $k^*$ in 0.808 of S1 runs; LEXIS without screening reached it in only 0.246, because it runs every opened null candidate to its cap. Adding screening (LEXIS-S) lifted this to 0.756 (S1) and 0.251 (S2, versus 0.269), but the preregistered non-inferiority margin of 0.05 was met in only 2 of 6 parameter settings, total episodes were unchanged (screening dominates cost), and scouting-weighted spending gave no detectable benefit. LEXIS-S was markedly more conservative (0.004 false confirmations versus 0.034): an anytime-valid test capped at a fixed sample uses less of its error budget, and that unused budget is the power it gives up. With 5,000 episodes and repeated passes both procedures reached $k^*$ almost always on S1 (0.998, 1.000) and in 0.733 versus 0.765 of S2 runs.

**Table 3:** Probability of reaching the true margin $k^* = 3$ at a matched single-pass budget and with repeated passes (2,000 runs per cell; slot level). Rows 1 and 3 come from the first comparison (E4), the others from the preregistered hybrid study (E8); repaired Algorithm 1 was measured in both (S1: 0.800 and 0.808).

| Procedure | S1 | S2 | Pr(false), boundary null |
|---|---|---|---|
| Algorithm 1 as specified (invalid) | 0.803 | 0.280 | 0.257 |
| Algorithm 1 repaired | **0.808** | **0.269** | 0.034 |
| LEXIS (no screening) | 0.246 | 0.043 | 0.018 |
| LEXIS-S (screening + e-process) | 0.756 | 0.251 | 0.004 |
| Algorithm 1 repaired, multi-pass (5,000) | 1.000 | 0.765 | 0.015 |
| LEXIS-S, multi-pass (5,000) | 0.998 | 0.733 | 0.014 |

### 8.5 Is a shared coordinator worth it?

No measurable speed-up. In all 20 Holm-corrected comparisons (two instances, $W \in \{2, \ldots, 32\}$) the shared-ledger deployment was indistinguishable from $W$ independent $\varepsilon/W$ deployments and from one centralized scheduler: restricted-mean-time ratios 0.77–1.30, every 95% CI containing 1, every adjusted $p \geq 0.94$. With 100 seeds per cell, effects smaller than a ratio of about 0.65 could not be detected. Sharing one budget spreads it over more registrations, roughly cancelling the union-bound saving, while frontier pruning saves only work above the current margin.

### 8.6 Certificates

Per-level certificates covered the true violating mass in every run (coverage 1.000 [0.981, 1.000]) but were loose: median bounds of 0.39–0.75 against true masses of 0–0.01 after 100–280 audits per level. They are valid statements about the generator, not evidence of robustness.

### 8.7 Real-agent experiments

Not yet run. The protocol: (1) profile 2,000 episodes per open-weight model to estimate timeout and crash rates separately for violating and benign trajectories, and measure redelivery rates of common queue configurations; (2) re-run the fault-injection studies of Tables 2 and the sequential ablations with the measured rates; (3) replay 200 completed slots to verify seeded determinism; (4) compare procedures on 20 MARGIN-Bench scenario families with cluster bootstrap over families; (5) report authorized-task completion on paired counterparts. Table 5 lists the required evidence.

**Table 4:** Preregistered hypotheses and verdicts (synthetic evidence).

| Hypothesis | Criterion | Verdict |
|---|---|---|
| H-valid | Wilson lower bound $\leq \varepsilon$ in all rule-compliant configurations | supported |
| H-rules | removing a rule gives lower bound $> \varepsilon$ (sequential, system level) | supported for 3 of 4 |
| H-alg1 | Algorithm 1 as specified exceeds $\varepsilon$; repair does not | supported |
| H-speed | speedup lower bound $\geq 16$ at $W = 32$ | supported (25.4×) |
| H-overhead | $\leq 10$ messages per episode, $\leq 100$ ledger ops/s | supported |
| H-recover | 90% throughput within 2× outage | supported |
| H-coop | shared ledger faster than independent or centralized | **not supported** |
| H-vs-Alg1 | LEXIS reaches $k^*$ as often as repaired Algorithm 1 | **refuted** |
| H6-noninf | LEXIS-S within 0.05 of repaired Algorithm 1 | **not supported** (2/6 settings) |
| H6-cheaper | LEXIS-S uses fewer episodes | **refuted** (ratio $\approx 1.00$) |
| H-cert | certificate coverage $\geq 0.95$ | supported; bounds loose |
| H-valid-real | measured fault profiles imply inflation | [TO RUN] |

**Table 5:** Real-agent result shells. No value may be filled except from executed runs.

| Quantity | Model A | Model B |
|---|---|---|
| Timeout/crash rate by outcome $(c_0, c_1)$ | [TO RUN] | [TO RUN] |
| Redelivery rate of the naive queue | [TO RUN] | [TO RUN] |
| Implied false-confirmation inflation per default | [TO RUN] | [TO RUN] |
| Replay determinism (% trace-hash match) | [TO RUN] | [TO RUN] |
| Episodes to $\hat{b}$: repaired Algorithm 1 / LEXIS-S | [TO RUN] | [TO RUN] |
| Authorized-task completion | [TO RUN] | [TO RUN] |

# 9 Practitioner checklist

1. Derive each episode's randomness from (run key, candidate, slot index); reveal it only when the slot is dispatched; publish a hash of the run key before the run.

2. Store results write-once per (candidate, slot) and ignore duplicates; never count a redelivered result.

3. Decide which slots form the evidence *before* running them: the in-order prefix for sequential tests, a fixed index set for fixed-sample tests. Never use "the first $n$ results to arrive".

4. Record a crashed or timed-out episode as non-violating when confirming (violating when certifying absence); never silently re-run it with new randomness. Replay with the same key only after auditing determinism; disable automatic retries in task frameworks.

5. Spend one error budget across all workers, all budget levels and all passes; correct across levels.

6. Recompute every confirmation from stored records.

7. Reset all agent state (memory, tool state) between episodes, and audit the oracle for false positives.

8. Preregister deadlines, thresholds and test parameters, and report imputation rates per arm.

## 10 Discussion and failure analysis

**What the evidence supports.** Ordinary execution semantics can multiply the error of agent security evaluations, sometimes to certainty, and six cheap rules prevent it for both common families of confirmation tests. The rules cost one write per episode, scale near-linearly, and recover quickly from faults. The audit shows the issue is not hypothetical bookkeeping: a concrete, carefully specified benchmark estimator was invalid for the related reason of uncorrected multiplicity.

**What it does not support.** A shared coordinator is not needed for speed: independent deployments with a split budget are as fast and equally valid, provided each follows the rules. Anytime-valid confirmation is not more sample-efficient: at a fixed budget the repaired fixed design is slightly more powerful, and early stopping saves little when screening dominates cost. Sequential confirmation remains the right choice when runs must be extended or stopped unpredictably.

**When the guarantees fail.** Shared hidden state between episodes (e.g. agent memory not reset) violates Assumption 1 and can inflate error under any execution; oracle false positives shift the guarantee to the observed rate; leaked keys enable selection on seeds. If violating episodes time out more often than benign ones, conservative imputation hides them: validity holds but power collapses, so deadlines are statistical parameters to be preregistered.

## 11 Threats to validity

*Construct:* the fault rates are chosen triggers, not measurements; real harnesses may show smaller (or larger) effects. *Internal:* procedure parameters were not tuned beyond a small preregistered sweep; one LEXIS variant is labelled exploratory; the fixed-sample rule study was added during reframing. *Statistical:* 400 runs per system-level cell limit detection of modest inflation, which is why the global-budget result remains open at $W = 8$; seeds share random streams across methods. *External:* simulated generators are non-adaptive and instances small. *Implementation:* the database-backed ledger is specified but not executed; coordination cost is measured in virtual time.

## 12 Limitations and negative results

(i) No real-agent results. (ii) No speed-up from a shared coordinator. (iii) Sequential confirmation, with or without screening, is somewhat less powerful than a repaired fixed design at a fixed budget. (iv) Certificates are loose at realistic audit budgets. (v) The necessity of the global budget was not established at $W = 8$ at system level for sequential tests (it was for fixed-sample tests). (vi) Byzantine executors are not covered. (vii) Bit-reproducible epoch mode and a database adapter are not implemented.

## 13 Reproducibility and data availability

Code, configurations, raw results (JSON lines with configuration hashes and code revisions), analysis scripts and figures are in the project repository [INSERT PERMANENT ARCHIVE DOI]. One script regenerates every result in about two hours on four cores; 46 unit, property-based and integration tests cover the ledger, protocol, procedures and simulator; the simulator is deterministic given configuration and seed.

## 14 Future work

Execute the real-agent protocol; measure the rules' effect in widely used open-source evaluation harnesses; extend the rules to estimators with adaptive allocation of confirmation trials and to Byzantine executors with attestation; and narrow the power gap of anytime-valid confirmation at fixed budgets.

## 15 Conclusion

The error level of an agent security evaluation is a property of the whole experiment, including how its harness delivers, retries, orders and budgets episodes. We gave six rules that make sequential and fixed-sample confirmation valid under the faults of real infrastructure, showed that each common default can break validity, repaired an estimator that was invalid as specified, and found that the rules are cheap while a shared coordinator and anytime-valid confirmation buy flexibility rather than speed or power. Real-agent measurements remain to be done.

## Declarations

**Funding.** [INSERT TRUE FUNDING INFORMATION OR "No funding was received".]
**Competing interests.** [AUTHORS TO DISCLOSE.]
**Ethics approval.** Not applicable to the simulation study; the real-agent protocol uses sandboxed mock tools and no human participants [CONFIRM BEFORE REAL-AGENT RUNS].
**Data availability.** Synthetic results are released with the code [ARCHIVE DOI]. No real-agent data exist yet.
**Code availability.** [REPOSITORY URL AND ARCHIVE DOI].
**Author contributions.** [CRediT STATEMENT].
**Use of generative AI.** [AUTHORS MUST DISCLOSE: substantial parts of the design, code, analysis and text of this working manuscript were drafted with an AI assistant and must be verified and declared according to the journal's policy.]

## References

[1] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. doi: 10.1145/3605764.3623985.

[2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks Track*, 2024. URL https://arxiv.org/abs/2406.13352. arXiv:2406.13352.

[3] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv:2403.02691, 2024. URL https://arxiv.org/abs/2403.02691.

[4] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM agents against long-horizon attacks. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://proceedings.mlr.press/v306/jiang26as.html.

[5] John Hughes et al. Best-of-N jailbreaking. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. URL https://arxiv.org/abs/2412.03556. arXiv:2412.03556.

[6] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv:2411.00640, 2024. URL https://arxiv.org/abs/2411.00640.

[7] Bandana Kaur. What AI red-team evaluations can and cannot prove. arXiv:2607.21735, 2026. URL https://arxiv.org/abs/2607.21735.

[8] MARGIN-Bench: Counterfactual safety margins for multi-turn and multi-agent large language model systems. Working manuscript v1.0, 18 October 2026 (unpublished), 2026.

[9] Aaditya Ramdas, Peter Grünwald, Vladimir Vovk, and Glenn Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4), 2023. doi: 10.1214/23-STS894.

[10] Vladimir Vovk and Ruodu Wang. E-values: Calibration, combination and applications. *The Annals of Statistics*, 49(3), 2021. doi: 10.1214/20-AOS2020.

[11] Peter Grünwald, Rianne de Heide, and Wouter Koolen. Safe testing. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 2024. doi: 10.1093/jrsssb/qkae011.

[12] Ian Waudby-Smith and Aaditya Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1), 2024. doi: 10.1093/jrsssb/qkad009.

[13] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2), 2021. doi: 10.1214/20-AOS1991.

[14] Ramesh Johari, Pete Koomen, Leonid Pekelis, and David Walsh. Always valid inference: Continuous monitoring of A/B tests. *Operations Research*, 70(3), 2022. doi: 10.1287/opre.2021.2135.

[15] Ruodu Wang and Aaditya Ramdas. False discovery rate control with e-values. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 84(3), 2022. doi: 10.1111/rssb.12489.

[16] Ziyu Xu and Aaditya Ramdas. Online multiple testing with e-values. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238 of *Proceedings of Machine Learning Research*, pages 3997–4005, 2024. URL https://proceedings.mlr.press/v238/xu24a.html.

[17] Dean P. Foster and Robert A. Stine. $\alpha$-investing: a procedure for sequential control of expected false discoveries. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 70(2), 2008. doi: 10.1111/j.1467-9868.2007.00643.x.

[18] Christopher R. Genovese, Kathryn Roeder, and Larry Wasserman. False discovery control with p-value weighting. *Biometrika*, 93(3), 2006. doi: 10.1093/biomet/93.3.509.

[19] Lisa V. Hampson and Christopher Jennison. Group sequential tests for delayed responses (with discussion). *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 75(1), 2013. doi: 10.1111/j.1467-9868.2012.01030.x.

[20] Marcel Böhme. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology*, 27(2), 2018. doi: 10.1145/3210309.

[21] Marcel Böhme, Danushka Liyanage, and Valentin Wüstholz. Estimating residual risk in greybox fuzzing. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021. doi: 10.1145/3468264.3468570.

[22] Håkan L. S. Younes and Reid G. Simmons. Statistical probabilistic model checking with a focus on time-bounded properties. *Information and Computation*, 204(9), 2006. doi: 10.1016/j.ic.2006.05.002.

[23] Axel Legay, Benoît Delahaye, and Saddek Bensalem. Statistical model checking: An overview. In *Runtime Verification (RV 2010)*, volume 6418 of *Lecture Notes in Computer Science*. Springer, 2010. doi: 10.1007/978-3-642-16612-9_11.

[24] Marco C. Campi and Simone Garatti. The exact feasibility of randomized solutions of uncertain convex programs. *SIAM Journal on Optimization*, 19(3), 2008. doi: 10.1137/07069821X.

[25] Hideaki Kano, Junya Honda, Kentaro Sakamaki, Kentaro Matsuura, Atsuyoshi Nakamura, and Masashi Sugiyama. Good arm identification via bandit feedback. *Machine Learning*, 2019. doi: 10.1007/s10994-019-05784-4.

[26] Andrea Locatelli, Maurilio Gutzeit, and Alexandra Carpentier. An optimal algorithm for the thresholding bandit problem. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *Proceedings of Machine Learning Research*, 2016. URL https://proceedings.mlr.press/v48/locatelli16.html.

[27] Maryam Aziz, Jesse Anderton, Emilie Kaufmann, and Javed Aslam. Pure exploration in infinitely-armed bandit models with fixed-confidence. In *Proceedings of Algorithmic Learning Theory (ALT)*, volume 83 of *Proceedings of Machine Learning Research*, pages 3–24, 2018. URL https://proceedings.mlr.press/v83/aziz18a.html.

[28] Arghya Roy Chaudhuri and Shivaram Kalyanakrishnan. PAC identification of a bandit arm relative to a reward quantile. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017. URL https://ojs.aaai.org/index.php/AAAI/article/view/10802.

[29] Eshcar Hillel, Zohar Karnin, Tomer Koren, Ronny Lempel, and Oren Somekh. Distributed exploration in multi-armed bandits. In *Advances in Neural Information Processing Systems 26 (NeurIPS)*, 2013. URL https://papers.nips.cc/paper/2013/hash/598b3e71ec378bd83e0a727608b5db01-Abstract.html.

[30] Liam Li, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzur, Moritz Hardt, Benjamin Recht, and Ameet Talwalkar. A system for massively parallel hyperparameter tuning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/a06f20b349c6cf09a6b171c71b88bbfc-Abstract.html.

[31] DynamiQ: Unlocking the potential of dynamic task allocation in parallel fuzzing. arXiv:2510.04469, 2025. URL https://arxiv.org/abs/2510.04469.

[32] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP)*, 1989. doi: 10.1145/74850.74870.

[33] Patrick E. O'Neil. The escrow transactional method. *ACM Transactions on Database Systems*, 11(4), 1986. doi: 10.1145/7239.7265.

[34] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *Lecture Notes in Computer Science*. Springer, 2011. doi: 10.1007/978-3-642-24550-3_29.

[35] Joseph M. Hellerstein and Peter Alvaro. Keeping CALM: When distributed consistency is easy. *Communications of the ACM*, 63(9), 2020. doi: 10.1145/3369736.

[36] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 1978. doi: 10.1145/359545.359563.

[37] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1), 1985. doi: 10.1145/214451.214456.

[38] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 1969. doi: 10.1137/0117039.

[39] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. URL https://www.usenix.org/conference/osdi18/presentation/moritz.

[40] Tao Luo, Mingen Pan, Pierre Tholoniat, Asaf Cidon, Roxana Geambasu, and Mathias Lécuyer. Privacy budget scheduling. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021. URL https://www.usenix.org/conference/osdi21/presentation/luo.

[41] TAMAS: Benchmarking adversarial risks in multi-agent LLM systems. arXiv:2511.05269, 2025. URL https://arxiv.org/abs/2511.05269.

[42] The dark side of LLMs: Agent-based attack vectors for system-level compromise. arXiv:2507.06850, 2025. URL https://arxiv.org/abs/2507.06850.

[43] Mantas Mazeika et al. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, 2024. URL https://proceedings.mlr.press/v235/mazeika24a.html.

[44] Tianzhuo Yang et al. AgentBoundary: Counterfactual evaluation of safety in tool-using LLM agents. arXiv:2609.33658, 2026. URL https://arxiv.org/abs/2609.33658.

[45] Viraaji Mothukuri and Reza M. Parizi. Causal detection of multi-step LLM agent attacks. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://proceedings.mlr.press/v306/mothukuri26a.html.

[46] Edoardo Debenedetti et al. Defeating prompt injections by design. arXiv:2503.18813, 2025. URL https://arxiv.org/abs/2503.18813.

[47] Tobin South et al. Authenticated delegation and authorized AI agents. arXiv:2501.09674, 2025. URL https://arxiv.org/abs/2501.09674.

[48] Ruoming Pang et al. Zanzibar: Google's consistent, global authorization system. In *2019 USENIX Annual Technical Conference (USENIX ATC)*, 2019. URL https://www.usenix.org/conference/atc19/presentation/pang.

[49] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. doi: 10.14722/ndss.2014.23212.

[50] Derek Lilienthal and Sanghyun Hong. Mind the gap: Time-of-check to time-of-use vulnerabilities in LLM-enabled agents. NeurIPS 2025; arXiv:2508.17155, 2025. URL https://arxiv.org/abs/2508.17155.

[51] Edward L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 1958. doi: 10.1080/01621459.1958.10501452.